

Product Requirements Document (PRD)

Project: KMSurveyTool Migration to Laravel

1. Executive Summary

KMSurveyTool is a comprehensive survey creation, management, and analysis platform designed to serve various user personas, including administrators, organizations, independent researchers, and respondents. The primary purpose of the application is to facilitate the seamless creation of public and invitation-only surveys, data collection, and result analysis.

The goal of this project is to migrate the existing flat-file PHP architecture into a modern, robust, and secure Laravel 11 application using the MVC paradigm. The application will leverage Laravel's built-in features for authentication, routing, ORM (Eloquent), and templating (Blade).

2. User Roles & Permissions

The system employs a strict Role-Based Access Control (RBAC) model with four primary tiers:

- **Admin (admin):** Superuser access. Can manage all users across all tiers, moderate surveys universally, oversee payments, and access system-wide analytics/reports.
- **Organization (organization):** Business users who can create and manage surveys for their company, track responses, and manage their subscription and payment statuses.
- **Independent Researcher (independent):** Academic/PhD users similar to Organizations but with specific profile requirements (e.g., institution, research area). They manage their own surveys and subscriptions.
- **Respondent (respondent):** Standard users who register to participate in targeted invitation-only surveys and track their submission history.
- **Guest (Public):** Unregistered users who can browse and take publicly available surveys.

3. Functional Requirements

Authentication & Onboarding

- Registration and Login flows customized per user role.
- Account status tracking (Pending/Active) with admin approval workflows where applicable.

Dashboard & Workspaces

- Role-specific dashboards displaying relevant metrics (e.g., total surveys, total responses, subscription status).

Survey Management

- **Survey Builder:** A visual drag-and-drop form builder interface (using SurveyJS or jQuery FormBuilder) to create complex surveys saving as JSON schema.
- **Legacy Builder Support:** Ability to manually construct questions (text, radio, matrix, geo, etc.) and save them relationally.
- **Survey Settings:** Configure title, description, category (Marketing, Academic, Product, Political), and visibility (Public vs. Invitation).

Survey Execution & Collection

- Dynamic rendering of the survey based on the saved JSON schema or relational questions.
- Responsive interface allowing respondents to easily submit answers on mobile and desktop.

Data Analysis & Reporting

- Aggregated reports for survey creators viewing collected data (charts, maps, text responses).

Subscriptions & Payments

- Integration gateways allowing Organizations and Independent Researchers to upgrade their subscription from "unpaid" to paid.

4. Data Model / Schema

Proposed Laravel Eloquent models and their relationships, derived from the existing SQLite database.

- **User**
 - *Columns:* id, name, email, password, role (enum), status (enum: pending/active), timestamps.
 - *Relations:* hasOne (Organization), hasOne (Independent), hasMany (Response).
- **Organization**
 - *Columns:* id, user_id, name, payment_status, subscription_expiry, timestamps.
 - *Relations:* belongsTo (User), hasMany (Survey), hasMany (Payment).
- **Independent**
 - *Columns:* id, user_id, name, institution, research_area, payment_status, subscription_expiry, timestamps.
 - *Relations:* belongsTo (User), hasMany (Survey), hasMany (Payment).
- **Survey**
 - *Columns:* id, organization_id (nullable), independent_id (nullable), title, description, category, type (enum: public/invitation), status (enum: draft/active), json_schema (longtext), created_by, timestamps.

- **Relations:** belongsTo (Organization), belongsTo (Independent), hasMany (Question), hasMany (Response).
- **Question** (Optional/Legacy support)
 - **Columns:** id, survey_id, text, type, options (json), required (boolean), position, timestamps.
 - **Relations:** belongsTo (Survey), hasMany (Answer).
- **Response**
 - **Columns:** id, survey_id, respondent_id (nullable for guests), timestamps.
 - **Relations:** belongsTo (Survey), belongsTo (User, 'respondent_id'), hasMany (Answer).
- **Answer**
 - **Columns:** id, response_id, question_id (nullable for JSON surveys), value (text/json), timestamps.
 - **Relations:** belongsTo (Response), belongsTo (Question).
- **Payment**
 - **Columns:** id, organization_id (nullable), independent_id (nullable), amount, method (enum: paypal/intasend), status, transaction_id, timestamps.
 - **Relations:** belongsTo (Organization), belongsTo (Independent).

5. Routes & Endpoints

Web Routes (Protected by Middleware per role)

- **Public:**
 - / (Landing Page)
 - /login, /register/{role}
 - /surveys/public (List public surveys)
 - /surveys/{id}/take (Take survey view)
- **Admin (/admin/*):**
 - /dashboard, /users, /surveys, /reports
- **Organization (/organization/*):**
 - /dashboard, /surveys/create, /surveys/{id}/edit, /responses
- **Independent (/independent/*):**
 - /dashboard, /surveys/create, /surveys/{id}/edit, /responses
- **Respondent (/respondent/*):**
 - /dashboard, /surveys/invitations, /responses/history

API Endpoints (JSON/REST for Survey Builder & Submissions)

- GET /api/surveys/{id} - Fetch survey JSON schema.
- POST /api/surveys/{id} - Save/update survey JSON schema.

- `POST /api/surveys/{id}/submit` - Submit respondent answers.
- `GET /api/user/surveys` - List surveys owned by the authenticated user.

6. Third-Party Integrations

- **Survey Generation UI:** FormBuilder.online (jQuery Form Builder) / SurveyJS Designer.
- **CSS Framework:** Current system uses W3.CSS. *Recommendation for Laravel: migrate to Tailwind CSS or Bootstrap via Vite.*
- **Icons:** FontAwesome.
- **Payment Gateways:** PayPal, IntaSend (for African payment methods like M-Pesa).
- **JavaScript Libraries:** jQuery, jQuery UI.

7. Non-Functional Requirements

- **Framework & Environment:** Laravel 11.x, PHP 8.2+, Composer.
- **Database:** SQLite for local dev/testing, MySQL or PostgreSQL for production.
- **Security:**
 - Implement Laravel Sanctum or standard session-based Auth.
 - All forms must have CSRF protection (`@csrf`).
 - Input validation using Laravel Form Requests.
 - Middleware for strict route protection and role verification ensuring cross-tenant data isolation.
- **Performance:**
 - Blade template caching.
 - Database indexing on frequently queried columns (user emails, survey ids).
 - Asset bundling with Vite.
- **Architecture:** Follow strict MVC. Controllers must remain thin, offloading complex business logic (like survey JSON parsing) to Service classes.